Apache Kafka

Uvod

Apache Kafka je open-source distribuirana event streaming platforma.

Distribuirana:

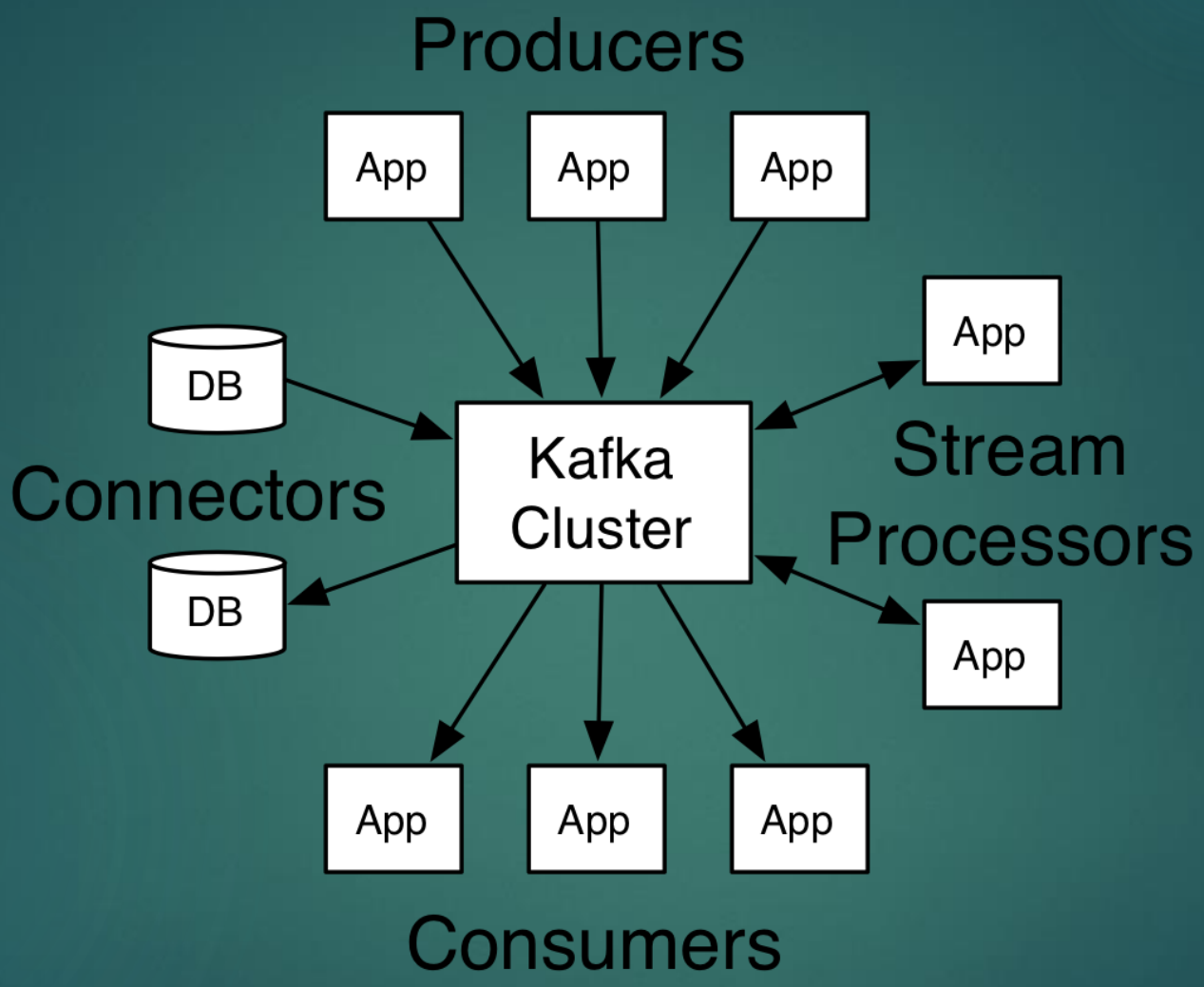
- Ima više servera (čvorova)
- Visoka dostupnost
- Skalabilnost

Event streaming znači neprekidno:

- prikupljanje
- pohranjivanje
- obrada i dijeljenje evenata

Sastoji se od:

- Servera - Jedna instanca Kafke ima jedan ili više servera pokrenutih u više data centara ili cloud regija
- Klijenata - Distribuirane aplikacije i mikroservisi (Java, JavaScript, Scala, Go, Python, C/C++)


Producers
App
App
App
DB
Connectors
DB
Kafka
Cluster
App
Stream
Processors
App
App
App
App
Consumers

Primjena

- Messaging
- Praćenje aktivnosti na web stranicama
- Praćenje metrika
- Log agregaciju
- Stream procesiranje

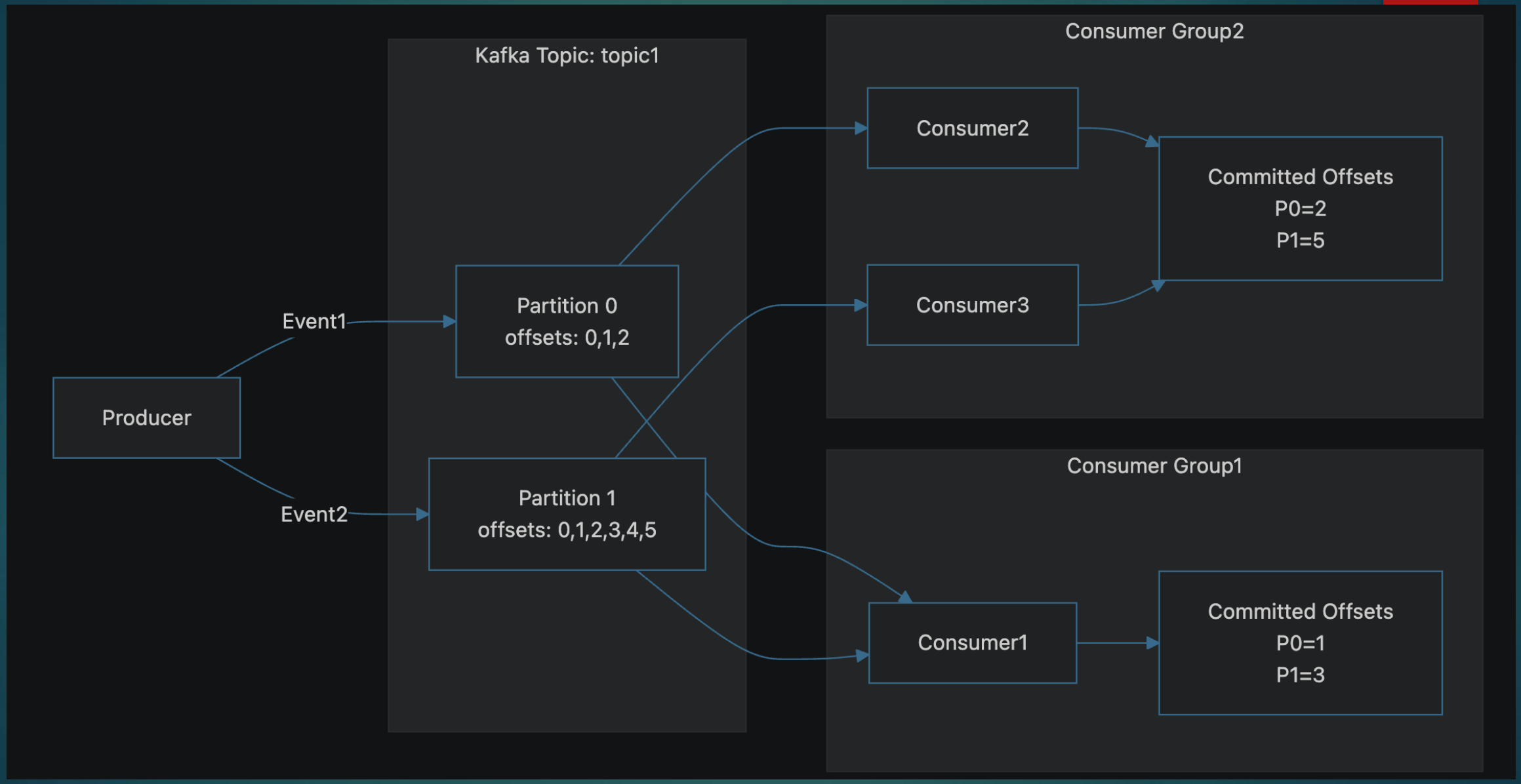
Glavni koncepti i terminologija

Event (događaj/zapis/poruka) bilježi činjenicu da se nešto dogodilo.

Event ima **key**, **value** i **timestamp**, te opcionalno metadata u **headeru**.

Primjer:

- Event key: “Hrvoje”
- Event value: “Uplatio 200 eur na Marinin račun.”
- Event timestamp: “Jun. 25, 2020 at 2:06 p.m.”


Kafka Topic: topic1
Consumer Group2
Consumer2
Committed Offsets
P0=2
P1=5
Consumer3
Event1
Partition 0
offsets: 0,1,2
Producer
Event2
Partition 1
offsets: 0,1,2,3,4,5
Consumer Group1
Consumer1
Committed Offsets
P0=1
P1=3

Producer

- šalje evente na Kafku
- batch.size (u bajtovima)

Consumer

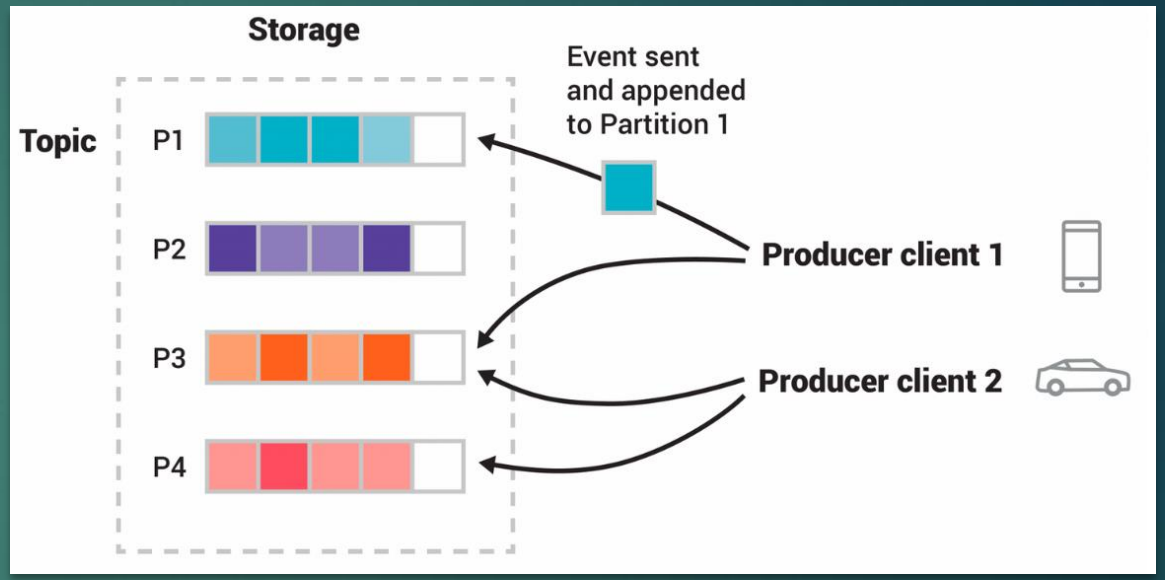
- čita i procesira evenate
- max.poll.records

Topic

- način na koji su organizirani eventi
- direktorij (message log)
- multi-producer i multi-subscriber
- eventi se ne brišu nakon konzumacije

Particija

- Dio topica
- Bucket
- Eventi s istim ključem se zapisuju u istu particiju
- Eventi jedne particije se zapisuju i čitaju slijedno



Repliciranje se radi na razini particija.

Leader Kad producer pošalje event na topic, šalje ga na leader particije. Isto je i kad consumer čita event.

Follower je broker koji ima repliku leader podataka.

Leader epoch je broj koji se povećava svakom promjenom leadera particije.

Event key

- Određuje u koju particiju će event biti poslan
- Pomoću njega se izgenerira hash koji eventu određuje particiju
- Čuva redoslijed
- Ako ključ nije definiran, koristi se round-robin algoritam

Offset

- sekvencijalni ID (broj)
- pozicija eventa u particiji

Commited offset – zadnji offset kojeg je consumer pročitao.

Consumer konfiguracija:

- enable.auto.commit – po defaultu true
- auto.commit.interval.ms – po defaultu 5 sekundi

Consumer group

- Za paralelnu obradu stream-a evenata, ili da bi dva consumer-a isti stream obrađivali na različit način
- Pamti commited offset za svaku particiju
- Consumeri u istoj grupi imaju isti group.id

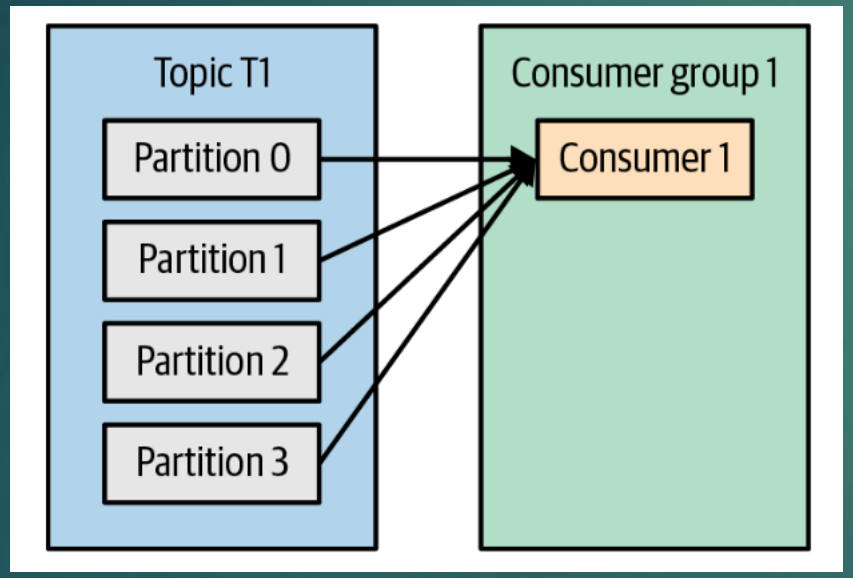
Jedna particija se može dodijeliti samo jednom consumeru po grupi u danom trenutku.

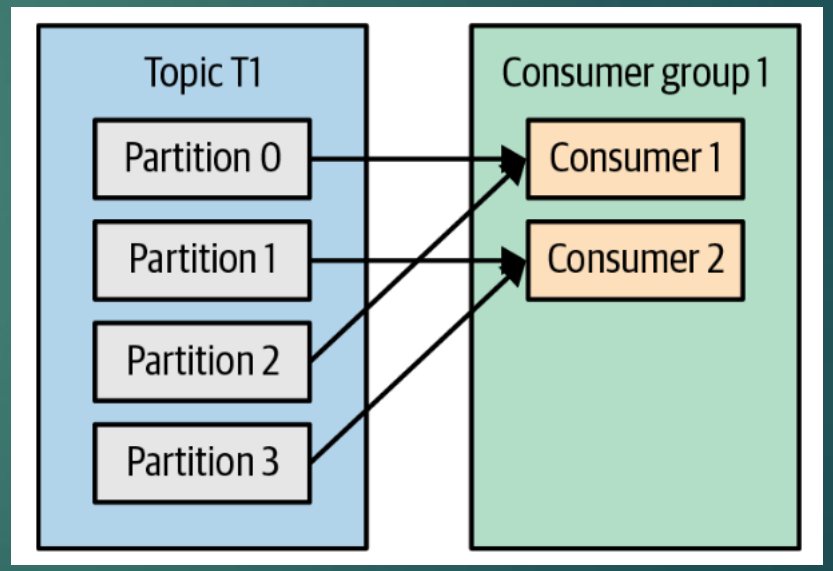
Konfiguracija **auto.offset.reset** (**latest** je default vrijednost) određuje kako će se consumer ponašati kad inicijalno ne postoji commited offset.

Rebalansiranje

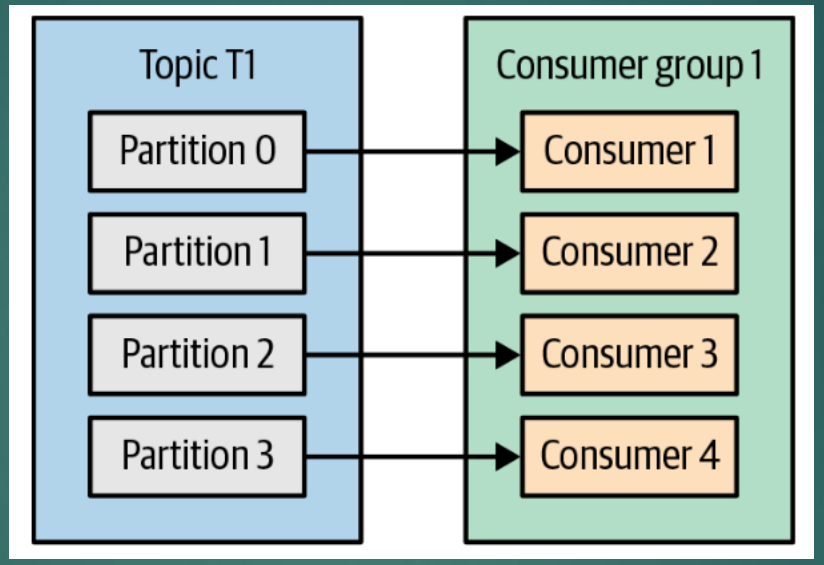
- Kad jednu particiju uzmemo jednom i damo drugom consumeru
- Consumeri šalju heartbeat

Dodavanje novog consumera:

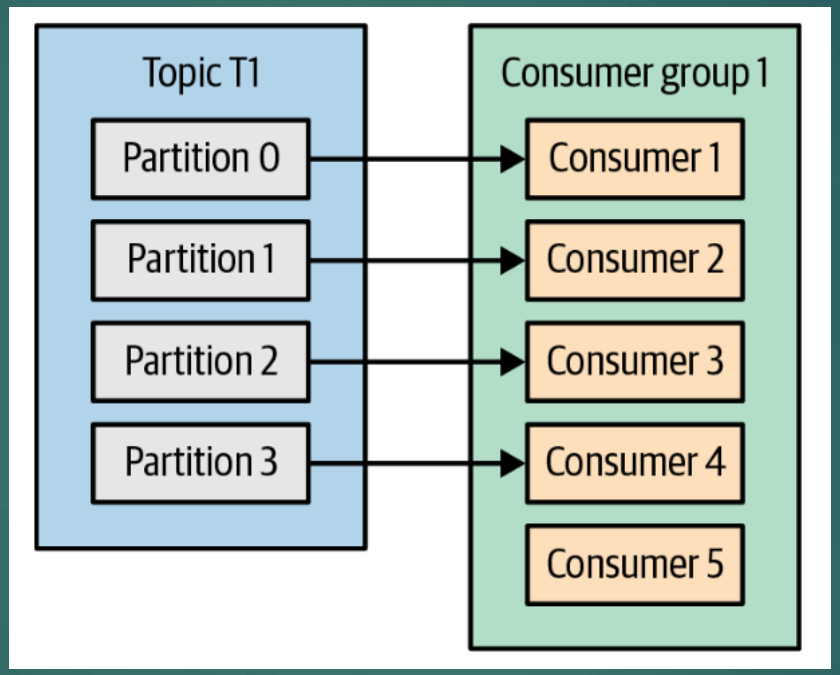




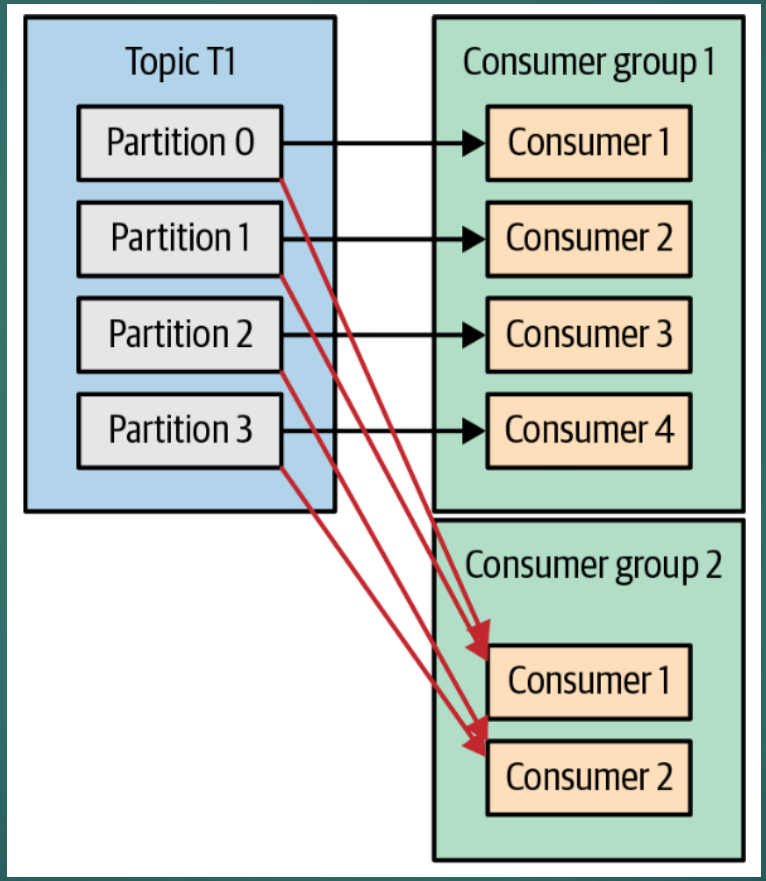
Ako dodamo još dva:



Dodajemo još jednog:



Svaka grupa će dobiti sve poruke:



Zadatak: Sustav za online narudžbe

Nacrtaj Kafka arhitekturu gdje:

- korisnici kreiraju narudžbe (event: `OrderCreated`)
- više servisa obrađuje narudžbe (payment, inventory, shipping)
- koristi se jedan topic `orders`

Prikaži: producer, topic, consumer.

Objasni što se događa s commited offsetima.

Zadatak: Streaming logova iz mikroservisa

Nacrtaj Kafka arhitekturu za sustav gdje svi mikroservisi šalju logove u Kafka topic `logs`.

Prikaži: 3 mikroservisa kao producere, topic (s particijama i offsetima), consumer

Pokaži što se događa kad želimo skalirati Kafku zbog povećanja broja pristiglih logova.

Prikaži što se događa ako jedan consumer padne.

Zadatak: Sustav za plaćanja s retry logikom

- producer šalje `PaymentRequested`
- consumer obrađuje plaćanja
- neuspješni event se ponovno šalje u isti topic

Prikaži producer, topic (s particijama i offsetima), consumer i objasni što će se dogoditi u slučaju neuspješne obrade eventa. Što ovdje moramo konfigurirati?

Zadatak: Sustav sa striktno sekvencijalnim izvršavanjem

Zamisli sustav za obradu bankovnih naloga gdje je redoslijed izvršavanja kritičan. Jedan servis obrađuje sve pristigle evente.

Imamo evente:

- MoneyDeposited
- MoneyWithdrawn
- BalanceUpdated

Prikaži: producer, event, topic (particije s offsetima), consumer.

Nacrtaj Kafka arhitekturu u kojoj se osigurava da se eventi obrađuju strogo redoslijedom kojim su poslani u slučaju kad imamo više klijenata banke.

Zadatak: Fraud Detection sustav

U sustavu postoje sljedeći događaji:

- UserLoggedIn
- PasswordChanged
- PaymentRequested

Događaji se šalju u zasebne Kafka topic-e: logins, security-events, payments.

Fraud Detection Service mora istovremeno pratiti sva tri topic-a kako bi mogao prepoznati sumnjive situacije.

Prikaži: 3 producera, 3 topic-a (s 1 particijom i offsetima), 1 consumer koji čita iz više topic-a, committed offsete.

Nacrtaj još jednu arhitekturu s 2 consumera.

Kafka APIs

- Admin API – za upravljanje i istraživanje
- Producer API – za publishiranje
- Consumer API – za subscribe
- Streams API –za obrađivanje event streamova
- Connect API – za import ili export stream-ova evenata iz vanjskih sustava

Instalacija

```bash
# Docker
docker compose up -d
```

Consumer i producer API

```bash
# Pokreni Docker shell
docker exec --workdir //opt/kafka -it kafka sh

# Kreiraj topic edukacija-events
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-topics.sh --create --bootstrap-server localhost:9092 \
--topic edukacija-events
```

```bash
# Ispiši sve topic-e
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--list


# Vidi detalje edukacija-events topic-a
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--describe --topic edukacija-events

# Brisanje topic-a
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--delete --topic edukacija-events
```

ISR (In-Sync Replicas) - to su replike koje su potpuno sinkronizirane s leaderom i imaju iste podatke kao i on.

Postavke:

- min.insync.replicas (na razini topica) - Određuje mininmalni broj replika koji mora acknowledge-ati write request kako bi se smatrao uspješnim.
- replica.lag.time.max.ms (na razini brokera) – Definira maksimalno vrijeme trajanja za koje se jedna follower replica mora uskladiti s leaderom prije nego se označi da je “out of sync”.

Lista parametara koji se mogu konfigurirati:

https://kafka.apache.org/44/configuration

```bash
# Kreiraj evente
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
--topic edukacija-events

# Kreiraj consumere
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--topic edukacija-events --from-beginning

docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--topic edukacija-events \
--command-property group.protocol=consumer

# Ispiši sve grupe
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
--list
```

```bash
# Vidi detalje grupe
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
--describe --group <ime grupe>

# Brisanje grupe
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
--delete --group <ime-grupe>
```

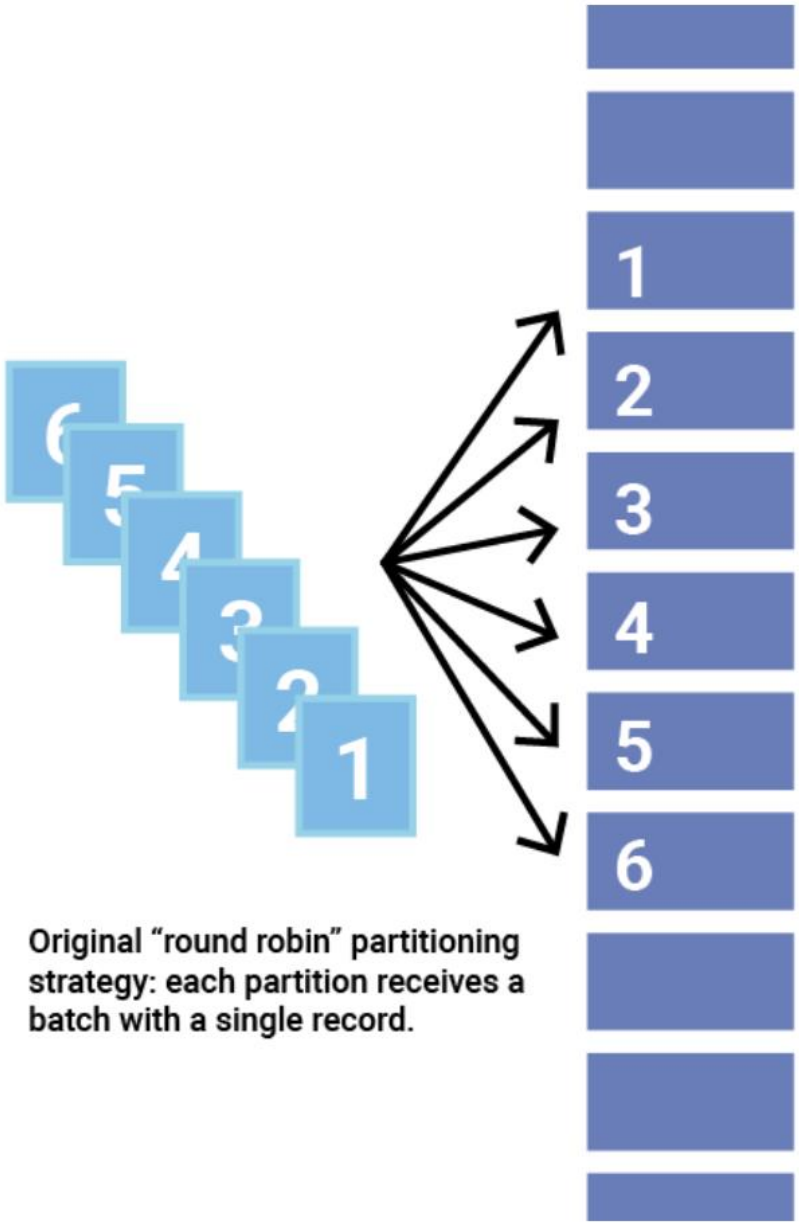
```bash
# Kreiraj producera
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
--topic edukacija-events

# Kreiraj consumera u grupi grupa-a koji će dobiti sve do sad poslane evente
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--topic edukacija-events --group grupa-a --from-beginning \
--command-property group.protocol=consumer

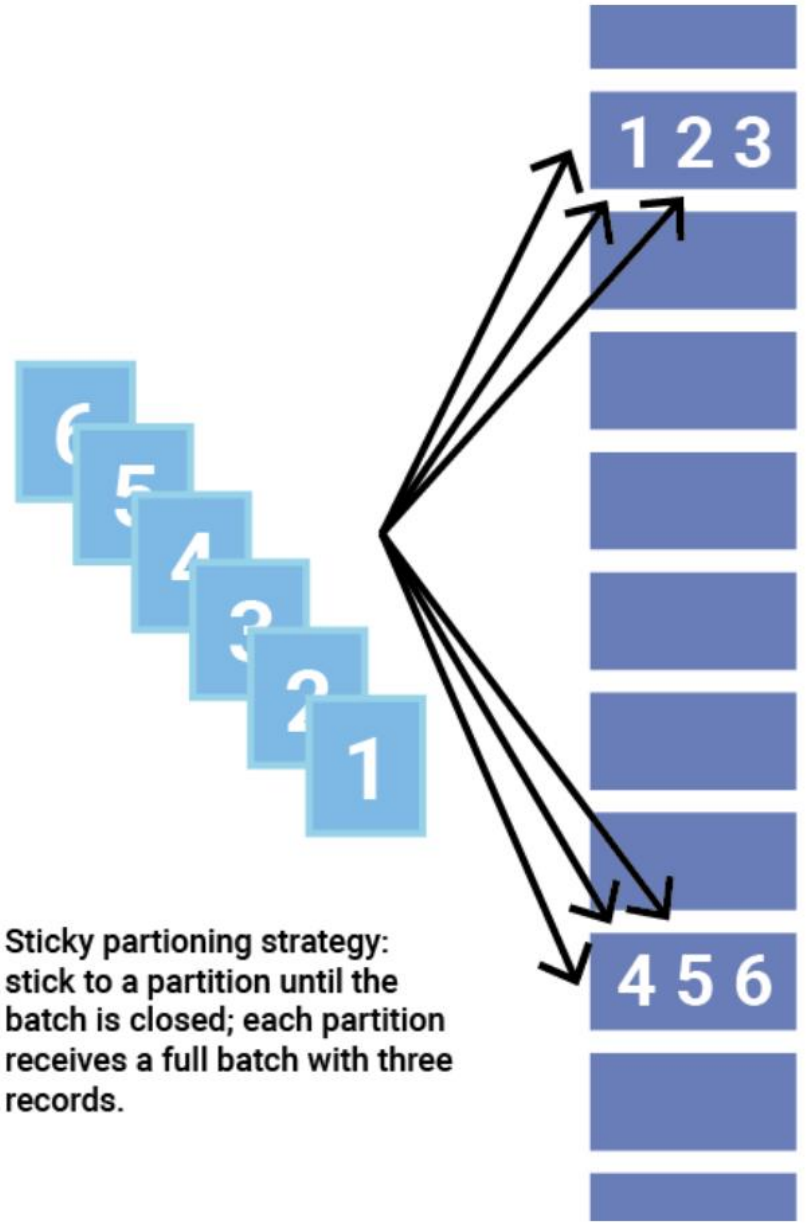
# Kreiraj dodatnog consumera u grupi grupa-a
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--topic edukacija-events --group grupa-a \
--command-property group.protocol=consumer
```

```bash
# Web UI
docker run -d --rm --name kafka-ui --network kafka-net \
-p 8080:8080 \
-e DYNAMIC_CONFIG_ENABLED=true \
-e KAFKA_CLUSTERS_0_NAME=local \
-e KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:19092 \
kafbat/kafka-ui:latest

# Otvori localhost:8080
```



Original “round robin” partitioning strategy: each partition receives a batch with a single record.



Sticky partioning strategy: stick to a partition until the batch is closed; each partition receives a full batch with three records.

```bash
# Kreiraj producera koji može poslati key
docker exec --workdir //opt/kafka -it kafka sh bin/kafka-console-producer.sh \
--bootstrap-server localhost:9092 \
--topic edukacija-events --reader-property parse.key=true --reader-property key.separator=:

# Kreiraj consumer-a koji čita key i vrijednost
docker exec --workdir //opt/kafka -it kafka sh \
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--command-property group.protocol=consumer \
--topic edukacija-events --group grupa-a \
--formatter-property print.key=true \
--formatter-property print.value=true
```

```bash
# Pročitaj poruke iz datoteke myevents.txt i pošalji na topic
cat myevents.txt | docker exec --workdir //opt/kafka -i kafka sh \
bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
--topic edukacija-events
```

Zadaci

1. Kreiraj 3 consumera bez definirane grupe i pošalji sve evente iz datoteke myevents.txt. Jesu li svi consumeri dobili poruku? Zašto?
2. Kreiraj 2 consumera s istom grupom i jedan bez definirane grupe. Jesu li svi consumeri dobili poruku? Zašto?
3. Kreiraj 3 consumera s istom gupom. Jesu li svi consumeri dobili poruku? Zašto?
4. Zatvori postojeće consumere i kreiraj dva nova kojima ćeš poslati sve evente iz gornje datoteke. Eventi jedne osobe moraju biti obrađeni u istom redoslijedu u kojem su poslani. Svaki event mora biti obrađen samo jednom.
5. Što je potrebno napraviti da svaki event bude obrađen dva puta, ali u istom redoslijedu u kojem se šalju?

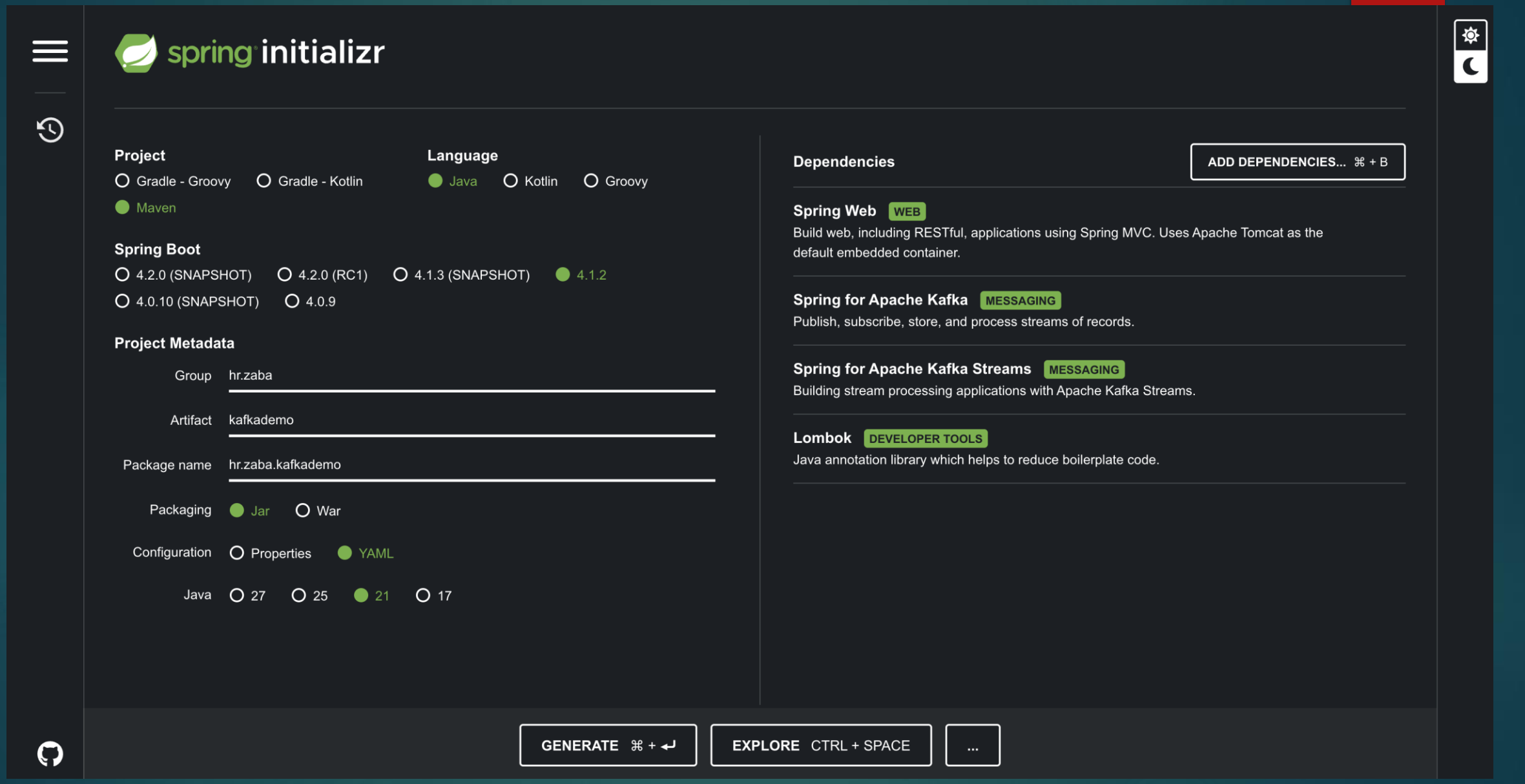
Ponavljanje

- Koliko evenata jedan producer može poslati odjednom na jednu particiju?
- Koliko evenata jedan consumer može pročitati odjednom s jedne particije?
- Na koliko particija jednog topica može biti spojen jedan consumer?
- Na koliko topica jedan consumer može biti spojen?
- Koliko consumera iz jedne grupe može biti spojeno na istu particiju?
- Što je presjek dviju consumer grupa?
- Koliko maksimalno commited offseta može imati jedan topic?
- Što je rebalansiranje? Kad se događa?
- Kako se sve postiže paralelna obrada evenata?
- Kako se topic replicira?

Zadaci

1. Implementiraj “sustav za online narudžbe” iz poglavlja “Glavni koncepti i terminologija”.
2. Implementiraj “streaming logova iz mikroservisa” iz poglavlja “Glavni koncepti i terminologija”. Evente šalji pomoću txt datoteke.
3. Implementiraj “sustav sa striktno sekvencijalnim izvršavanjem” iz poglavlja “Glavni koncepti i terminologija”. Evente šalji pomoću txt datoteke.

Spring Boot


spring initializr
Project
Gradle - Groovy
Gradle - Kotlin
Maven
Language
Java
Kotlin
Groovy
Spring Boot
4.2.0 (SNAPSHOT)
4.2.0 (RC1)
4.1.3 (SNAPSHOT)
4.1.2
4.0.10 (SNAPSHOT)
4.0.9
Project Metadata
Group
hr.zaba
Artifact
kafkademo
Package name
hr.zaba.kafkademo
Packaging
Jar
War
Configuration
Properties
YAML
Java
27
25
21
17
Dependencies
ADD DEPENDENCIES... ⌘ + B
Spring Web
WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
Spring for Apache Kafka
MESSAGING
Publish, subscribe, store, and process streams of records.
Spring for Apache Kafka Streams
MESSAGING
Building stream processing applications with Apache Kafka Streams.
Lombok
DEVELOPER TOOLS
Java annotation library which helps to reduce boilerplate code.
GENERATE ⌘ + ↵
EXPLORE CTRL + SPACE
...

```bash
# Kreiraj topic
docker exec --workdir //opt/kafka -it kafka \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--create \
--replication-factor 1 \
--partitions 1 \
--topic mytopic

# Kreiraj consumera
docker exec --workdir //opt/kafka -it kafka \
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--command-property group.protocol=consumer \
--topic mytopic --group grupa-a
```

```application.yaml
server:
        port: 8081

spring:
        kafka:
                bootstrap-servers: localhost:9092
```

```Java
@RestController
@RequestMapping("/kafka")
public class KafkaTestController {

	@Autowired
	private KafkaTemplate<String, String> kafkaTemplate;

	@GetMapping("/test")
	public String test() {
		var poruka = "Moj prvi Spring Boot event";
		// kafkaTemplate.send(MY_TOPIC, poruka);

		// Ako zelimo cekati odgovor asinkrono
		CompletableFuture<SendResult<StriKafkaTestControllerng, String>> future = kafkaTemplate.send("mytopic", poruka);
		future.whenComplete((result, ex) -> {
			if (ex == null) {
				System.out.println("Sent message=[" + poruka +
				"] with offset=[" + result.getRecordMetadata().offset() + "]");
			} else {
				System.out.println("Unable to send message=[" +
				poruka + "] due to : " + ex.getMessage());
			}
		});

		return "Ok";
	}
}
```

```Java
@Component
public class KafkaConsumerConfig {

	@KafkaListener(
			topics = { "mytopic" },
			groupId = "foo",
			concurrency = "1",
			properties = {
				ConsumerConfig.MAX_POLL_RECORDS_CONFIG + "=100"
		})
		public void listenWithHeaders(
			@Payload String message,
			@Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
			@Header(KafkaHeaders.OFFSET) int offset) throws Exception {
				log.debug("Received Message: " + message
				+ "\n from partition: " + partition + ", group: foo, offset: " + offset);
		}
}
```

```Java
@Component
public class KafkaConsumerConfig {

	@KafkaListener(
		topicPartitions = @TopicPartition(topic = "mytopic", partitionOffsets = { @PartitionOffset(partition = "0", initialOffset = "6") })
	)
	public void listenToPartition(
		@Payload String message,
		@Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
		@Header(KafkaHeaders.OFFSET) int offset) {
			log.debug("Received Message: " + message + "\n from partition: " + partition + ", offset: " + offset);
	}
}
```

```Java
@Bean
public ConsumerFactory<Long, Order> orderConsumerFactory() {
	Map<String, Object> config = new HashMap<>();

	config.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, this.bootstrapAddress);
	config.put(ConsumerConfig.GROUP_ID_CONFIG, "orders-group");
	config.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, LongDeserializer.class);
	config.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonDeserializer.class);

	config.put(JsonDeserializer.TRUSTED_PACKAGES, "*");

	return new DefaultKafkaConsumerFactory<>(config);
}

@Bean
public ConcurrentKafkaListenerContainerFactory<Long, Order> ordersKafkaListenerContainerFactory() {
	ConcurrentKafkaListenerContainerFactory<Long, Order> factory =
		new ConcurrentKafkaListenerContainerFactory<>();
	factory.setConsumerFactory(orderConsumerFactory());
	return factory;
}
```

```Java
@Component
public class KafkaConsumerConfug {

	@KafkaListener(
		topics = "orders",
		groupId = "orders-group",
		containerFactory = "ordersKafkaListenerContainerFactory"
	)
	public void listenOrders(Order order, @Header(KafkaHeaders.RECEIVED_PARTITION) int partition) {
		log.debug("Received order: " + order + " from partition: " + partition);
	}
}
```

Zadaci

1. Kreiraj novi topic “orders” s 3 particije.
2. Kreiraj Kafka listenera i šalji JSON poruke oblika: { "customerId": "123", "product": "MacBook", "amount": 1500 }
3. Pošalji poruku s ključem, koristi customerId kao Kafka ključ.
4. Pošalji 10+ poruka s 2-3 različita customerId-a i u listeneru logiraj particiju i ključ. Potvrdi da sve poruke istog customerId-a uvijek idu na istu particiju.

Kafka Streams API

- Deklarativno obrađivanje stream-a poruka
- Ima “Processor Topology”, tj. sastoji se od mreže čvorova (procesora) kroz koje podaci prolaze

kstream biblioteka

- stateful i stateless procesiranje
- podržava razne transformacije podataka: filtriranje, mapiranje, flat-mapping, agragiranje, join

KTable i KStream API

KStream API

- stateless - procesira podatke
- ključevi nisu bitni

KTable API

- stateful - pamti i ažurira stanje
- bitno imaju li podaci isti ključ
- sprema key-value podatke u RocksDB local store i Kafka changelog topic

```java
// Serializers/deserializers (serde) for String and Long types
final Serde<String> stringSerde = Serdes.String();
final Serde<Long> longSerde = Serdes.Long();

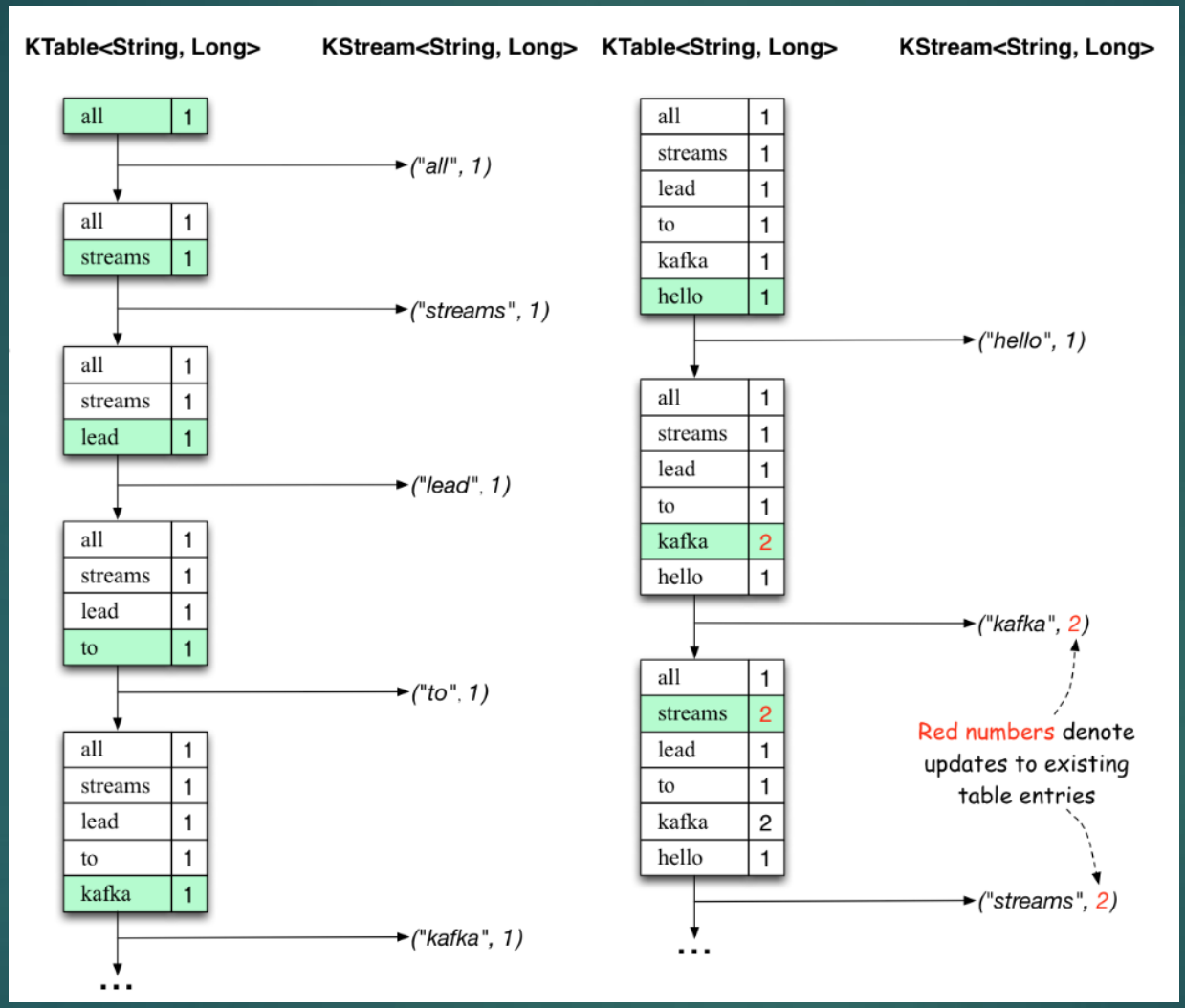
// Construct a `KStream` from the input topic "streams-plaintext-input", where message values
// represent lines of text (for the sake of this example, we ignore whatever may be stored
// in the message keys).
KStream<String, String> textLines = builder.stream(
      "streams-plaintext-input",
      Consumed.with(stringSerde, stringSerde)
    );

KTable<String, Long> wordCounts = textLines
    // Split each text line, by whitespace, into words.
    .flatMapValues(value -> Arrays.asList(value.toLowerCase().split("\\W+")))

    // Group the text words as message keys
    .groupBy((key, value) -> value)

    // Count the occurrences of each word (message key).
    .count();

// Store the running counts as a changelog stream to the output topic.
wordCounts.toStream().to("streams-wordcount-output", Produced.with(Serdes.String(), Serdes.Long()));
```


KTable<String, Long>
KStream<String, Long>
KTable<String, Long>
KStream<String, Long>
all 1
("all", 1)
all 1
streams 1
("streams", 1)
all 1
streams 1
lead 1
("lead", 1)
all 1
streams 1
lead 1
to 1
("to", 1)
all 1
streams 1
lead 1
to 1
kafka 1
("kafka", 1)
...
all 1
streams 1
lead 1
to 1
kafka 1
hello 1
("hello", 1)
all 1
streams 1
lead 1
to 1
kafka 2
hello 1
("kafka", 2)
all 1
streams 2
lead 1
to 1
kafka 2
hello 1
Red numbers denote updates to existing table entries
("streams", 2)
...

```bash
# Kreiraj input topic
docker exec --workdir //opt/kafka/ -it kafka sh \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--create \
--replication-factor 1 \
--partitions 1 \
--topic streams-plaintext-input


# Kreiraj output topic
docker exec --workdir //opt/kafka/ -it kafka sh \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--create \
--replication-factor 1 \
--partitions 1 \
--topic streams-wordcount-output


# Vidi detalje kreiranih topic-a
docker exec --workdir //opt/kafka/ -it kafka sh \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--describe --topic streams-wordcount-output
```

```bash
# Pokreni WordCount aplikaciju
docker exec --workdir //opt/kafka/ -it kafka sh \
bin/kafka-run-class.sh org.apache.kafka.streams.examples.wordcount.WordCountDemo

# Pokreni producer za input topic
docker exec --workdir //opt/kafka/ -it kafka sh \
bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
--topic streams-plaintext-input

# Pokreni consumer-a
docker exec --workdir //opt/kafka/ -it kafka sh \
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--command-property group.protocol=consumer \
--topic streams-wordcount-output \
--from-beginning \
--formatter-property print.key=true \
--formatter-property print.value=true \
--formatter-property key.deserializer=org.apache.kafka.common.serialization.StringDeserializer \
--formatter-property value.deserializer=org.apache.kafka.common.serialization.LongDeserializer
```

SpringBoot implementacija

```Java
@Configuration
@EnableKafka
@EnableKafkaStreams
public class KafkaStreamsConfig {

	@Value(value = "${spring.kafka.bootstrap-servers}")
	private String bootstrapAddress;

	private static final String APP_ID = "testStreams";

	private static final Serde<String> STRING_SERDE = Serdes.String();

	@Bean(name = KafkaStreamsDefaultConfiguration.DEFAULT_STREAMS_CONFIG_BEAN_NAME)
	public KafkaStreamsConfiguration kStreamsConfig() {
		Map<String, Object> props = new HashMap<>();
		props.put(StreamsConfig.APPLICATION_ID_CONFIG, APP_ID);
		props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapAddress);
		props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG, STRING_SERDE.getClass().getName());
		props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, STRING_SERDE.getClass().getName());

		return new KafkaStreamsConfiguration(props);
	}
}
```

```Java
	@Bean
	public KStream<String, String> kStream(StreamsBuilder kStreamBuilder) {
		KStream<String, String> stream = kStreamBuilder.stream("my-input-streams-topic");
		KTable<String, Long> wordsCount = stream
		// .peek((key, value) -> System.out.println("Received: key=" + key + ", value=" + value))
		.mapValues((ValueMapper<String, String>) String::toLowerCase)
		// .peek((key, value) -> System.out.println("After mapValues: " + value))
		.flatMapValues(value -> Arrays.asList(value.split("\\W+")))
		// .peek((key, value) -> System.out.println("After flatMapValues: " + value))
		.groupBy((key, value) -> value)
		.count();

		wordsCount.toStream().to("my-output-streams-topic", Produced.with(STRING_SERDE, Serdes.Long()));
		return stream;
	}
```

```bash
# Kreiraj input streams topic
docker exec --workdir //opt/kafka -it kafka \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--create \
--replication-factor 1 \
--partitions 1 \
--topic my-input-streams-topic

# Kreiraj output streams topic
docker exec --workdir //opt/kafka -it kafka \
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--create \
--replication-factor 1 \
--partitions 1 \
--topic my-output-streams-topic

# Kreiraj producera
docker exec --workdir //opt/kafka -it kafka \
bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
--topic my-input-streams-topic \
--reader-property "parse.key=true" \
--reader-property "key.separator=:"
```

Custom SerDes

Potrebno implementirati:

- org.apache.kafka.common.serialization.Serializer i
- org.apache.kafka.common.serialization.Deserializer

Postoje već implementacije za Avro, Protobuf i JSON.

```Java
Serdes.serdeFrom(new JsonSerializer<>(), new JsonDeserializer<>(Order.class));

KStream<String, Order> ordersStream = kStreamBuilder
        .stream("orders",
                Consumed.with(Serdes.String(),
                Serdes.serdeFrom(new JsonSerializer<>(), new JsonDeserializer<>(Order.class)))
        );
```

Topologija procesora

Tipovi procesora:

- Source – konzumira podatke iz source topika.
- Stream – obogaćuje, filtrira i transformira podatke.
- Sink – zapisuje podatke natrag u Kafku.

```java
	@Autowired
	private StreamsBuilderFactoryBean factoryBean;

	@GetMapping(value = "/topology", produces = MediaType.TEXT_PLAIN_VALUE)
	public String getTopology() {
		return factoryBean.getTopology().describe().toString();
	}
```

https://zz85.github.io/kafka-streams-viz

Zadaci

Kreiraj stream procesor kojem je ulazna vrijednost JSON poruka koja stiže na Kafka topic orders: {”customerId": "abc-123", "amount": 250.0}

Od te poruke trebaju nastati dva izlaza:

1. processed-orders - svaka narudžba čiji je amount veći od 100 dobiva prefiks: "Processed Order: {" customerId ": "abc-123", "amount": 250.0}”
2. aggregated-orders - za svaki customerId (koji je Kafka ključ poruke) akumulira se ukupni iznos. Dakle ako stignu dvije poruke s istim ključem:

   key: "abc-123", amount: 250.0

   key: "abc-123", amount: 80.0 ← ova se filtrira jer je < 100

   key: "abc-123", amount: 150.0

   Na izlazu za abc-123 bit će 400.0 (samo 250 + 150, jer 80 ne prođe filter).

Transakcije

- koraci procesiranja (konzumiranje, state store ažuriranja i slanje na output topice) su dio jedne atomic transakcije

```java
        props.put(StreamsConfig.PROCESSING_GUARANTEE_CONFIG, StreamsConfig.EXACTLY_ONCE_V2);
```

Join

Možemo spojiti:

- Stream-stream
- Stream-table
- Table-table

Koristi ključ evenata i definirani vremenski prozor za spajanje streamova.

ValueJoiner interface prima vrijednosti i kreira novu.

message.timestamp.type – može biti CreateTime ili LogAppendTime

Vrste Joinova

- Inner – emitira samo ako su obje strane dostupne unuter vremenskog prozora
- Outer – uvijek emitira nešto
- Left-outer – emitira nešto ako je dostupna lijeva strana

Primjer

```java
ValueJoiner<Order, Order, Order> valueJoiner = (leftValue, rightValue) -> {
        Order ret = new Order();
        ret.setOrderId(leftValue.getCustomerId());
        ret.setAmount(leftValue.getAmount() + rightValue.getAmount());
        return ret;
};

KStream<String, Order> joinedStream = leftStream
.peek((key, value) -> log.debug("Before join: key=" + key + ", value=" + value))
.join(rightStream,
        valueJoiner,
        JoinWindows.ofTimeDifferenceWithNoGrace(Duration.ofSeconds(5)),
        StreamJoined.with(
                Serdes.String(),
                this.orderSerde(),
                this.orderSerde()
        )
);

joinedStream.to("join-topic", Produced.with(STRING_SERDE, this.orderSerde()));
```

Zadaci

Sustav dobiva podatke za poziciju taksista na driver-topic i poziciju s koje je taksistu poslan zahtjev za voznju na ride-topic.

Ako su lokacije tasksista i klijenta dovoljno blizu, posalji ride-topic event na active-requests topic.

```json
// driver-topic event:
{
  "driverId": "d-1",
  "lat": 45.81,
  "lon": 15.97
}
```

```json
// ride-topic event:
{
  "rideId": "r-1",
  "driverId": "d-1",
  "lat": 45.81,
  "lon": 15.97
}
```

Branching

- Omogućuje da jedan strim razdvojimo u više novih

```java
ordersStream
        .split()
        .branch((key, value) -> value.getAmount() > 100, Branched.withConsumer((ks) -> ks.to("valid-orders")))
        .branch((key, value) -> value.getAmount() <= 100, Branched.withConsumer((ks) -> ks.to("invalid-orders")))
        .noDefaultBranch();
        /*.defaultBranch(
                Branched.withConsumer(ks ->
                ks.to("transactions-unknown")
                )
        );*/
```

```java
iotDevice
        .split()
        .branch((key, value) -> value.getSensorType() != null,
        Branched.withConsumer(ks -> ks.to((key, value, recordContext) -> "%s_%s".formatted(iotTopicName, value.getSensorType()))))
```

Zadaci

Napravi Streams procesor koja prima bankovne transakcije s transactions-input topica i automatski ih raspoređuje u različite Kafka topice ovisno o polju status.

Pretpostavimo da imamo tri statusa: APPROVED, REJECTED, PENDING.

- APPROVED → transactions-approved
- REJECTED → transactions-rejected
- PENDING → transactions-pending

```json
// transaction-input event:
{
 "transactionId": "tx-1001",
 "accountId": "acc-55",
 "amount": 250.75,
 "status": "APPROVED"
}
```

U transaction-unknown slati nepoznate statuse.

Connect API

Vrste connectora

1. Source – import podataka u Kafku
2. Sink - eksport

```bash
# Kopiraj u KafkaRadionica Kafka konfiguracijske datoteke
docker cp kafka:/opt/kafka/config .

# Ugasi Kafka servis
docker compose down
```

Dodaj u docker-compose.yaml:

```yaml
volumes:
        - ~/dev/Edukacije/KafkaRadionica:/home/appuser/shared:rw
        - ~/dev/Edukacije/KafkaRadionica/config:/opt/kafka/config:rw
```

```sh
# Ponovo pokreni Kafka servis
docker compose up -d
```

U config/connect-standalone.properties dodati:

```txt
plugin.path=libs/connect-file-4.3.1.jar
```

U config/connect-file-source.properties postaviti file putanju:

```txt
file=/home/appuser/shared/test.txt
```

U config/connect-file-sink.properties postaviti file putanju:

```txt
file=/home/appuser/shared/test.sink.txt
```

```bash
# Pokreni connectore
docker exec --workdir //opt/kafka -it kafka sh \
bin/connect-standalone.sh config/connect-standalone.properties config/connect-file-source.properties config/connect-file-sink.properties
```

Zadaci

1. Kreiraj dva nova consumera koji će dobiti sve evente importane preko connectora. Svaki event mora biti obrađen samo jednom.
2. Dodaj nove evente u source datoteku i vidi što će se dogoditi.

Zadaci

Podesi da WordCounter stream procesor preko connectora čita evente iz test.txt datoteke i zapisuje rezultate u counter test.sink.txt.